fdtool

Theo S Schult – GRR20241982

November 2026

Repositório: `https://github.com/tss6/fdtool/`

1 Definições

Seja:

- R: um esquema relacional
- U: o universo de atributos de R
- F: um conjunto de dependências funcionais sobre U

Dependência Funcional (DF)
Para $X, Y \subseteq U$, X determina Y ($X \to Y$) se cada valor de X é associado com exatamente um valor Y.

DF Trivial
$X \to Y$ é trivial se $Y \subseteq X$.

Axiomas de Armstrong
Reflexividade (se $Y \subseteq X$ então $X \to Y$), aumento (se $X \to Y$ então $XZ \to YZ$) e transitividade (se $X \to Y$ e $Y \to Z$ então $X \to Z$).

Fecho de atributos
O fecho X^+ de X em relação a F é o maior conjunto $Y \subseteq U$ tal que $X \to Y \in F^+$.

Equivalência e cobertura
F e G são equivalentes ($F \equiv G$) se $F^+ = G^+$, ou seja, se toda DF de G é implicada por F e vice-versa.

Atributo estranho
Em $X \to A \in G$, o atributo $B \in X$ é estranho se $A \in (X - \{B\})^+$ em relação a G.

DF redundante
$X \to A \in G$ é redundante se $A \in X^+$ em relação a $G - \{X \to A\}$.

Cobertura mínima
G é uma cobertura mínima de F se $G \equiv F$, todo RHS de G tem um único atributo, nenhuma DF de G possui atributo estranho no LHS e nenhuma DF de G é redundante. Uma cobertura mínima não é única.

Superchave
$K \subseteq U$ é superchave se $K^+ = U$.

Chave candidata
Superchave minimal: $K^+ = U$ e, para todo $K' \subsetneq K$, $K'^+ \neq U$.

Atributo primo
Atributo que pertence a pelo menos uma chave candidata.

BCNF
R está na Forma Normal de Boyce-Codd se, para toda DF não trivial $X \to A$ de F^+, X é superchave.

3FN
R está na Terceira Forma Normal se, para toda DF não trivial $X \to A$ de F^+, X é superchave *ou* A é primo.

Se uma relação está em BCNF, também está em 3NF. A recíproca não vale.

2 Representação

2.1 Bitsets

Atributos foram restritos a `A..Z`, escolhi por representar um conjunto de atributos como um inteiro de 32 bits (`typedef uint32_t AttrSet`), onde o bit i determina se o atributo `'A'`$+i$ está presente. Dessa forma, as operações de conjunto são operações binárias (*bitwise*) de custo constante:

Operação	Implementação
$X \cup Y$	`x | y`
$X \cap Y$	`x & y`
$X - Y$	`x & ~y`
$X \subseteq Y$	`!(x & ~y)`
$\|X\|$	`__builtin_popcount(x)` (ou *Hamming weight*)

Uma DF é o par `struct FD { AttrSet lhs; AttrSet rhs; }` e uma instância (`Inst`) é o universo U mais um vetor dinâmico de DFs.

2.2 Formato `.fds`

Desenvolvi um *lexer* e um *parser* para a seguinte gramática que entendi ser a pretendida:

```
file   := decl decl
decl   := 'U' '=' '{' [ attr (sep attr)* ] '}'
        | 'F' '=' '{' [ fd   (sep fd)*   ] '}'
fd     := attr+ '->' attr+
attr   := 'A' .. 'Z'
sep    := ',' | ';'
```

Qualquer tipo de *whitespace* é ignorado. Ordem não importa. Erros e avisos são reportados no estilo do compilador GCC.

3 Algoritmos

Notação para a análise: $n = |U| \leq 26$, $m = |F|$ e m' é o número de DFs após a decomposição do RHS ($m' \leq n \cdot m$).

3.1 Fecho X^+

Algoritmo 1 Fecho de atributos (`closure` / `closure_skip`)

Entrada: conjunto X, DFs F
Saída: X^+ em relação a F

```
1:  mudou ← 1
2:  while mudou do
3:      mudou ← 0
4:      for i ← 0 até |F| − 1 do
5:          if lhs_i ⊆ X e rhs_i ⊈ X then
6:              X ← X ∪ rhs_i
7:              mudou ← 1
8:          end if
9:      end for
10: end while
11: return X
```

Explicação. Cada atributo é adicionado a X a partir de transitividade com o que já existe em X. Se existe uma DF_i onde podemos aplicar o axioma, unimos X com o rhs_i. O laço *while* termina se nenhuma DF atualizou X, significando que não é possível expandir X além.

Complexidade. Toda passagem que não é a última acrescenta pelo menos um atributo, então há no máximo $n + 1$ passagens de m testes cada: $O(n \cdot m)$ tempo e $O(1)$ memória extra. Adicionei a variante `closure_skip` que ignora uma DF pelo índice e é usada na remoção de DFs redundantes, evitando copiar o vetor.

3.2 Cobertura mínima

Algoritmo 2 Cobertura mínima (`min_cover`)

Entrada: instância (U, F)
Saída: cobertura mínima G com RHS unitário

```
1:  G ← ∅
2:  for all X → Y ∈ F do                          ▷ 1. RHS unitário
3:      for all A ∈ Y − X do
4:          G ← G ∪ {X → A}                       ▷ descarta a parte trivial
5:      end for
6:  end for
7:  for all X → A ∈ G do                          ▷ 2. atributos estranhos
8:      for all B ∈ X com |X| > 1 do
9:          if A ∈ (X − {B})^+ em relação a G then
10:             X ← X − {B}
11:         end if
12:     end for
13: end for
14: for all X → A ∈ G, em ordem do                ▷ 3. DFs redundantes
15:     if A ∈ X^+ em relação a G − {X → A} then
16:         remove X → A de G
17:     end if
18: end for
19: return G
```

Explicação.

- O passo 1 aplica a regra da decomposição e descarta os atributos de $Y \cap X$, que são triviais. Obviamente, o resultado é equivalente a F.
- No passo 2, se $A \in (X - \{B\})^+$ então $(X - \{B\}) \to A$ já é implicada por G, e trocar $X \to A$ por ela mantém a equivalência.
- No passo 3, uma DF implicada pelas demais pode ser removida sem alterar G^+. A remoção é feita uma DF por vez, recalculando o fecho sobre o G já reduzido, o que impede remover duas DFs que só são redundantes uma em relação à outra.

Complexidade. O passo 1 custa $O(n \cdot m)$ e gera m' DFs. O passo 2 faz até n fechos por DF: $O(m' \cdot n \cdot nm') = O(n^2 m'^2)$. O passo 3 faz um fecho e um deslocamento do vetor (`memmove`) por DF: $O(m' \cdot (nm' + m')) = O(n\, m'^2)$. Total: $O(n^2 m'^2)$, ou $O(n^4 m^2)$ em função da entrada. Memória: $O(m')$.

3.3 Chaves candidatas

Primeiro classificamos os atributos de U como em grafos direcionados, pela posição em que aparecem em DFs não triviais de F:

- **fonte** (S): não aparecem em nenhum RHS (vizinhança de entrada vazia). Não podem ser derivados, então pertencem a *toda* chave.
- **sumidouro**: aparecem só em RHS (vizinhança de saída vazia). Nunca pertencem a uma chave. Prova: se uma chave K contém um sumidouro Z, removê-lo não invalida nenhuma DF de ser usada, e Z continua sendo derivado, logo $(K - \{Z\})^+ = U$ e K não é minima, contradição.
- **meio** (M): aparecem em algum LHS e em algum RHS. São os únicos que precisam ser testados.

Toda chave candidata tem a forma $S \cup T$ com $T \subseteq M$. O algoritmo enumera os subconjuntos T de M em ordem numérica crescente da máscara de bits, que parte de $T = \emptyset$ e volta a $\emptyset$ após visitar todas as $2^{|M|}$ submáscaras.

Algoritmo 3 Chaves candidatas (`keys`)

Entrada: instância (U, F)
Saída: lista $\mathcal{K}$ de todas as chaves candidatas

1: $L \leftarrow \bigcup lhs_i$; $R \leftarrow \bigcup (rhs_i - lhs_i)$
2: $S \leftarrow U - R$; $M \leftarrow U \cap R \cap L$
3: $\mathcal{K} \leftarrow \emptyset$; $T \leftarrow \emptyset$
4: **repeat**
5: $\quad C \leftarrow S \cup T$
6: $\quad$ **if** $\nexists K \in \mathcal{K} : K \subseteq C$ **e** $C^+ = U$ **then**
7: $\quad\quad \mathcal{K} \leftarrow \mathcal{K} \cup \{C\}$
8: $\quad$ **end if**
9: $\quad T \leftarrow (T - M)\ \&\ M$ ▷ próxima submáscara
10: **until** $T = \emptyset$
11: **return** $\mathcal{K}$

Explicação (minimalidade). Se $T' \subsetneq T$, a máscara de T' é numericamente menor que a de T, então todo subconjunto próprio de um candidato é visitado antes dele. Quando C é testado, todas as chaves contidas em C já estão em $\mathcal{K}$. Se nenhuma está, nenhum subconjunto próprio de C é superchave e, se $C^+ = U$, C é chave candidata. Se alguma está, C é superchave não mínima e é descartado.

Complexidade. São $2^{|M|}$ candidatos. Cada um custa um teste de minimalidade $O(k)$, sendo k o número de chaves já encontradas, e um fecho $O(n \cdot m)$. Total: $O(2^{|M|}(nm + k))$, aceitável pois o número de chaves candidatas pode ser exponencial em n. Com $n \leq 26$ o pior caso é $2^{26} \approx 6{,}7 \times 10^7$ candidatos.

3.4 Verificação de BCNF e 3FN

Algoritmo 4 Verificação de formas normais (`nf_violations`)

Entrada: esquema (U, F), chaves candidatas $\mathcal{K}$, modo (BCNF ou 3FN)
Saída: lista de DFs que violam a forma normal

1: $G \leftarrow \{X \to A : X \to Y \in F,\ A \in Y - X\}$ ▷ RHS unitário, sem triviais
2: $P \leftarrow \bigcup_{K \in \mathcal{K}} K$ ▷ atributos primos
3: **for all** $X \to A \in G$ **do**
4: **if** $X^+ = U$ **then continue** ▷ X é superchave
5: **end if**
6: **if** modo = 3FN **e** $A \in P$ **then continue** ▷ A é primo
7: **end if**
8: registra violação $X \to A$
9: **end for**

Explicação. G é F decomposto em RHS unitário, sem as partes triviais (passo 1 da cobertura mínima). As violações listadas são, portanto, DFs escritas na entrada. O teste de superchave usa o fecho e o teste de primo usa a união das chaves candidatas.

Não é preciso examinar todo F^+. Suponha que uma DF não trivial $X \to A \in F^+$ viola a forma normal, ou seja, X não é superchave (e, na 3FN, A não é primo). Ao calcular X^+ com G, alguma $Y \to A \in G$ com $Y \subseteq X^+$ adiciona A. Se Y fosse superchave, $U = Y^+ \subseteq X^+$ e X seria superchave, contradição. Logo $Y \to A$ também viola a forma normal e é encontrada pelo algoritmo. O RHS unitário é necessário na 3FN: em $B \to CD$ com C primo e D não, só a parte $B \to D$ viola.

Complexidade. A decomposição custa $O(nm)$ e gera m' DFs. Há uma chamada a `keys` (feita uma vez e compartilhada entre os dois modos) e um fecho por DF de G ($O(m' \cdot nm)$). O custo é dominado por `keys`.

3.5 Resumo das complexidades

Operação	Tempo	Memória extra
Leitura e análise do `.fds`	O(tamanho do arquivo)	O(tamanho do arquivo)
`closure`	$O(nm)$	$O(1)$
`mincover`	$O(n^2 m'^2) \subseteq O(n^4 m^2)$	$O(m')$
`keys`	$O(2^{\lvert M\rvert}(nm + k))$	$O(k)$
`normalform`	$O(2^{\lvert M\rvert}(nm + k) + nmm')$	$O(m' + k)$

4 Estudos de caso

Notação: $\{X\} \xrightarrow{Y \to Z} \{XZ\}$ indica que a DF $Y \to Z$ foi aplicada no cálculo de um fecho, e X_G^+ é o fecho de X em relação a G. As tabelas de formas normais listam as DFs de F com RHS unitário, e P é o conjunto de atributos primos.

4.1 Caso 1: exemplo da especificação (violação de BCNF e 3FN)

Arquivo `exemplos/prof.fds`:

```
U = {A,B,C,D,E}
F = { A->BC, C->D, BD->E }
```

Fecho.

$$A^+ : \{A\} \xrightarrow{A \to BC} \{ABC\} \xrightarrow{C \to D} \{ABCD\} \xrightarrow{BD \to E} \{ABCDE\} = U$$
$$BD^+ : \{BD\} \xrightarrow{BD \to E} \{BDE\}$$

Cobertura mínima.

1. RHS unitário: $G = \{A \to B,\ A \to C,\ C \to D,\ BD \to E\}$.
2. Atributos estranhos. Só $BD \to E$ tem LHS com mais de um atributo:

   $$\text{sem } B : D_G^+ = \{D\} \not\ni E$$
   $$\text{sem } D : B_G^+ = \{B\} \not\ni E$$

   Nenhum atributo é estranho.
3. DFs redundantes. Para cada DF, fecho do LHS sem ela:

   $$A \to B : \{A\} \xrightarrow{A \to C} \{AC\} \xrightarrow{C \to D} \{ACD\} \not\ni B \quad \Rightarrow \text{mantém}$$
   $$A \to C : \{A\} \xrightarrow{A \to B} \{AB\} \not\ni C \quad \Rightarrow \text{mantém}$$
   $$C \to D : \{C\} \not\ni D \quad \Rightarrow \text{mantém}$$
   $$BD \to E : \{BD\} \not\ni E \quad \Rightarrow \text{mantém}$$

G não muda:

```
$ ./fdtool mincover --fds exemplos/prof.fds
A->B
A->C
C->D
BD->E
```

Chaves.

$$L = A \cup C \cup BD = \{ABCD\}$$
$$R = BC \cup D \cup E = \{BCDE\}$$
$$S = U - R = \{A\} \quad \text{(fonte)}$$
$$M = U \cap R \cap L = \{BCD\} \quad \text{(meio)}$$
$$R - L = \{E\} \quad \text{(sumidouro)}$$

O primeiro candidato é $S \cup \emptyset = \{A\}$, e $A^+ = U$ (fecho acima), então $\{A\}$ é chave. Os outros $2^3 - 1$ candidatos contêm $\{A\}$ e são descartados sem calcular fecho. $\mathcal{K} = \{A\}$ e $P = \{A\}$.

Formas normais.

$X \to A$	X^+	X superchave?	$A \in P$?	BCNF	3FN
$A \to B$	$\{ABCDE\} = U$	sim	não	✓	✓
$A \to C$	$\{ABCDE\} = U$	sim	não	✓	✓
$C \to D$	$\{CD\}$	não	não	×	×
$BD \to E$	$\{BDE\}$	não	não	×	×

```
$ ./fdtool normalform --fds exemplos/prof.fds
BCNF: VIOLATIONS
3NF: VIOLATIONS
VIOLATION BCNF: C->D (C not superkey)
VIOLATION BCNF: BD->E (BD not superkey)
VIOLATION 3NF: C->D (D not prime, C not superkey)
VIOLATION 3NF: BD->E (E not prime, BD not superkey)
```

$C \to D$ e $BD \to E$ são dependências transitivas da chave: $A \to C \to D$ e $A \to BD \to E$.

4.2 Caso 2: em 3FN, mas não em BCNF

Arquivo `exemplos/multkeys.fds`:

```
U = {A,B,C}
F = { AB->C, C->B }
```

Cobertura mínima.

1. RHS unitário: $G = F = \{AB \to C,\ C \to B\}$.
2. Atributos estranhos em $AB \to C$:

$$\text{sem } A : B^+_G = \{B\} \not\ni C$$
$$\text{sem } B : A^+_G = \{A\} \not\ni C$$

3. DFs redundantes:

$$AB \to C : \{AB\} \not\ni C \qquad \Rightarrow \text{mantém}$$
$$C \to B : \{C\} \not\ni B \qquad \Rightarrow \text{mantém}$$

G não muda. Saída: `AB->C`, `C->B`.

Chaves.

$$L = AB \cup C = \{ABC\}$$
$$R = C \cup B = \{BC\}$$
$$S = U - R = \{A\} \qquad \text{(fonte)}$$
$$M = U \cap R \cap L = \{BC\} \qquad \text{(meio)}$$
$$R - L = \emptyset \qquad \text{(sumidouro)}$$

Enumeração dos subconjuntos $T \subseteq M$ em ordem crescente de máscara:

T	$S \cup T$	Fecho	Resultado
$\emptyset$	$\{A\}$	$\{A\}$	não é superchave
$\{B\}$	$\{AB\}$	$\{AB\} \xrightarrow{AB \to C} \{ABC\} = U$	chave
$\{C\}$	$\{AC\}$	$\{AC\} \xrightarrow{C \to B} \{ABC\} = U$	chave
$\{BC\}$	$\{ABC\}$	–	contém $\{AB\}$, descartado

$\mathcal{K} = \{AB,\ AC\}$ e $P = AB \cup AC = \{ABC\} = U$.

Formas normais.

$X \to A$	X^+	X superchave?	$A \in P$?	BCNF	3FN
$AB \to C$	$\{ABC\} = U$	sim	sim	✓	✓
$C \to B$	$\{BC\}$	não	sim	×	✓

```
$ ./fdtool normalform --fds exemplos/multkeys.fds
BCNF: VIOLATIONS
3NF: OK
VIOLATION BCNF: C->B (C not superkey)
```

$C \to B$ viola a BCNF porque C não é superchave, mas satisfaz a 3FN porque B é primo.

4.3 Caso 3: redundância e atributo estranho

Arquivo `exemplos/extra.fds`:

```
U = {A,B,C,D}
F = { A->B, B->C, A->C, AB->D }
```

Fecho.

$$A^+ : \{A\} \xrightarrow{A \to B} \{AB\} \xrightarrow{B \to C} \{ABC\} \xrightarrow{AB \to D} \{ABCD\} = U$$
$$B^+ : \{B\} \xrightarrow{B \to C} \{BC\}$$

Cobertura mínima.

1. RHS unitário: todas as DFs já têm RHS unitário e nenhuma é trivial, então $G = F$.
2. Atributos estranhos. Só $AB \to D$ tem LHS com mais de um atributo:

   $$\text{sem } A : B_G^+ = \{BC\} \not\ni D$$
   $$\text{sem } B : A_G^+ = \{ABCD\} \ni D \qquad \Rightarrow B \text{ é estranho, } AB \to D \text{ vira } A \to D$$

   Agora $G = \{A \to B,\ B \to C,\ A \to C,\ A \to D\}$. O atributo B é estranho porque A já determina B.
3. DFs redundantes, em ordem. O fecho é calculado sobre o G já reduzido, sem a DF testada:

   $$A \to B : \{A\} \xrightarrow{A \to C} \{AC\} \xrightarrow{A \to D} \{ACD\} \not\ni B \qquad \Rightarrow \text{mantém}$$
   $$B \to C : \{B\} \not\ni C \qquad \Rightarrow \text{mantém}$$
   $$A \to C : \{A\} \xrightarrow{A \to B} \{AB\} \xrightarrow{B \to C} \{ABC\} \ni C \qquad \Rightarrow \text{remove}$$
   $$A \to D : \{A\} \xrightarrow{A \to B} \{AB\} \xrightarrow{B \to C} \{ABC\} \not\ni D \qquad \Rightarrow \text{mantém}$$

```
$ ./fdtool mincover --fds exemplos/extra.fds
A->B
B->C
A->D
```

Chaves.

$$
\begin{aligned}
L &= A \cup B \cup A \cup AB = \{AB\} \\
R &= B \cup C \cup C \cup D = \{BCD\} \\
S &= U - R = \{A\} && \text{(fonte)} \\
M &= U \cap R \cap L = \{B\} && \text{(meio)} \\
R - L &= \{CD\} && \text{(sumidouros)}
\end{aligned}
$$

O primeiro candidato é $S \cup \emptyset = \{A\}$, e $A^+ = U$ (fecho acima), então $\{A\}$ é chave. O único outro candidato, $\{AB\}$, contém $\{A\}$ e é descartado. $\mathcal{K} = \{A\}$ e $P = \{A\}$.

Formas normais.

$X \to A$	X^+	X superchave?	$A \in P$?	BCNF	3FN
$A \to B$	$\{ABCD\} = U$	sim	não	✓	✓
$B \to C$	$\{BC\}$	não	não	×	×
$A \to C$	$\{ABCD\} = U$	sim	não	✓	✓
$AB \to D$	$\{ABCD\} = U$	sim	não	✓	✓

```
$ ./fdtool normalform --fds exemplos/extra.fds
BCNF: VIOLATIONS
3NF: VIOLATIONS
VIOLATION BCNF: B->C (B not superkey)
VIOLATION 3NF: B->C (C not prime, B not superkey)
```

A tabela inclui $A \to C$ e $AB \to D$, embora a cobertura mínima remova a primeira e reescreva a segunda, pois a verificação usa as DFs da entrada. Ambas têm LHS superchave. $B \to C$ é uma dependência transitiva da chave: $A \to B \to C$.

5 Limitações

- **Universo limitado a 26 atributos** de uma letra (`A..Z`, sem minúsculas), o que foi o motivo principal de bitsets terem sido escolhidos em vez de grafos.
- **Custo exponencial de `keys`** no número de atributos do "meio" ($2^{|M|}$, até 2^{26}). As entradas tendem a ser pequenas devido a limitação do Universo, mas instâncias muito grandes podem demorar um pouco.
- **Cobertura mínima dependente da ordem** da entrada. Os arquivos `order1.fds` e `order2.fds` diferem apenas na ordem de $A \to C$ e $B \to C$ e produzem coberturas diferentes ($B \to C$ contra $A \to C$), ambas válidas. Ideal seria identificar e potencialmente imprimir todas as coberturas que a instância permite.
- **Deduplicação apenas na saída.** Por exemplo, na sequinte dependência: $U = \{A, B, C\}$ e $F = \{A \to B,\ B \to C,\ B \to C\}$, tudo funciona sem *warnings*, porém a saída imprime apenas uma vez cada FD única, como explicado no e-mail.
- **Sem decomposição**: a ferramenta diagnostica as formas normais, mas não propõe decomposições em BCNF ou 3FN.

6 Principais decisões

- **Bitset para conjuntos de atributos.** Foi considerada uma representação em grafo, mas com no máximo 26 atributos operações de conjuntos ficam simples e rápidas.
- **Analisador léxico e sintático** com base na gramática que esperada e diagnósticos no estilo do GCC. O subcomando extra, `check`, só valida o arquivo.
- **Enumeração crescente de submáscaras** em `keys`,o que garante minimalidade sem um passo extra de filtragem e reduz o espaço de busca.
- **Interface.** Opções aceitas como `--fds arq` ou `--fds=arq`, em qualquer ordem; `--X` aceita `AB`, `A,B` ou `A B` e é validado contra U. Códigos de saída: 0 (ok), 1 (erro de leitura) e 2 (uso incorreto).
- **Saída ordenada.** Conjuntos são impressos percorrendo os bits de `A` a `Z`, ordenando automaticamente.

7 Uso de IA e referências

Inteligência artificial foi utilizada para pesquisar tópicos e esclarecer conceitos, não para escrever código-fonte. A versão base deste relatório foi gerada com auxílio de IA a partir do código e dos exemplos do repositório, e depois revisada pelo autor.

- **Disciplina**
  - Slides da disciplina de Banco de Dados.
- **Dependências funcionais e cobertura mínima**
  - `https://www.geeksforgeeks.org/dbms/how-to-find-minimal-set/`
  - `https://stackoverflow.com/questions/10284004/`
- **Enumeração de submáscaras (chaves candidatas)**
  - `https://www.inf.ufpr.br/andre/Disciplinas/CI1031-2026-2/03-busca-completa-matematica.pdf`
- **Contagem de bits**
  - `https://stackoverflow.com/questions/109023/`
  - `https://www.felixcloutier.com/x86/popcnt`
  - `https://gcc.gnu.org/onlinedocs/gcc/Bit-Operation-Builtins.html#index-_005f_005fbuiltin_005fpopcount`
  - `https://clang.llvm.org/docs/LanguageExtensions.html#builtin-popcountg`
  - `https://learn.microsoft.com/en-us/cpp/intrinsics/popcnt16-popcnt-popcnt64?view=msvc-170`
  - `https://en.wikipedia.org/wiki/Hamming_weight`
- **Vetor dinâmico**
  - `https://www.youtube.com/watch?v=gtk3RZHwJUA`
  - `https://stackoverflow.com/questions/1100311/`
- **Analisador léxico e sintático**
  - `https://github.com/lotabout/write-a-C-interpreter/blob/master/tutorial/en/3-Lexer.md`
  - `https://stackoverflow.com/questions/17975913/`
  - `https://github.com/rexim/bnfuzzer`
- **Linguagem C**
  - `https://stackoverflow.com/questions/7762731/`
  - `https://sourceware.org/glibc/manual/latest/html_node/Feature-Test-Macros.html`
  - `https://gcc.gnu.org/onlinedocs/gcc/Common-Attributes.html#index-format`
  - `https://stackoverflow.com/a/35281548`
- **Cores no terminal**
  - `https://gist.github.com/JBlond/2fea43a3049b38287e5e9cefc87b2124`